

Ramanujan Dreams on NVIDIA H100

MareNostrum5 CUDA Code Benchmark

EuroHPC Project ehpc916 — 2000 Node-Hours on H100 GPUs

David Vesterlund

EuroHPC JU / Barcelona Supercomputing Center (BSC)
inc407859@bsc.es

September 18, 2026

Abstract

We present three CUDA implementations of the Conservative Matrix Field (CMF) walk algorithm for generalized hypergeometric functions ${}_pF_q$, targeting NVIDIA H100 GPUs on the MareNostrum5 supercomputer at Barcelona Supercomputing Center. The implementations use three distinct arithmetic strategies: native double precision (f64), Residue Number System (RNS) exact modular arithmetic, and double-single (df64) emulated extended precision. All implementations are validated against the reference `ramanujantools` Python library.

We benchmark all three variants across five CMF types (${}_3F_2$, ${}_4F_3$, ${}_5F_4$, ${}_6F_5$, ${}_8F_7$) with 10,000 trajectories each at walk depth $N = 1000$, using both random initial points (limit search) and the classical zeta initial points $(1, \dots, 1; 2, \dots, 2)$ with $z = 1$ (delta search). A key technical contribution is log-space scale tracking through projective normalization, enabling correct delta (irrationality measure) computation despite per-step rescaling of the walk matrix.

Results show that df64 achieves the highest throughput for limit search (up to 789 000 traj/s for ${}_3F_2$), while RNS provides exact arithmetic at $\sim 100\times$ lower throughput but with guaranteed correctness. For the standard Ramanujan Machine workflow of GPU-based limit sweeping followed by CPU-based delta refinement, the floating-point variants provide a **100–1000 $\times$ speedup** over single-threaded Python. GPU-based delta search is also shown to be viable, with throughputs of 100 000 – 10 000 000 traj/s depending on CMF rank.

Keywords: Conservative Matrix Fields, CUDA, GPU computing, hypergeometric functions, irrationality measures, residue number system, NVIDIA H100, EuroHPC, Ramanujan Machine

Contents

1	Introduction	3
1.1	Project Scope	3
2	Background	3
2.1	Conservative Matrix Fields	3
2.2	The Matrix Walk	4
2.3	Irrationality Measure (Delta)	4
2.4	Zeta Initial Points	4
3	Methods	4
3.1	Three CUDA Variants	4
3.1.1	Variant 1: Native Float64 (f64)	4
3.1.2	Variant 2: Residue Number System (RNS)	4
3.1.3	Variant 3: Double-Single (df64)	4
3.2	Log-Space Scale Tracking	5

3.3	Companion Matrix Optimization	5
3.4	Parallelization Strategy	5
3.5	Validation Against ramanujantools	5
4	Benchmark Configuration	5
5	Results	5
5.1	Limit Search Throughput	5
5.2	Validity: Random vs Zeta Initial Points	6
5.3	GPU Kernel Time and Scaling	6
5.4	Memory Usage	6
5.5	Speedup over RNS	6
5.6	Delta Search Throughput	6
5.7	Detailed Results Tables	7
6	Discussion	7
6.1	When to Use Each Variant	7
6.1.1	GPU Limit Sweeping (df64 or f64)	7
6.1.2	GPU Delta Search (New Capability)	7
6.1.3	Exact Verification (RNS)	8
6.2	Limitations	8
7	Resource Utilization	8
7.1	GPU Hours Used	8
7.2	Job Summary	9
8	Conclusion	9

1 Introduction

The Ramanujan Machine project [1] discovers mathematical constants through algorithmic exploration of Conservative Matrix Fields (CMFs) and Polynomial Continued Fractions (PCFs). A CMF defines a family of matrices satisfying a conservation property, enabling the computation of limits and irrationality measures (deltas) for generalized hypergeometric functions ${}_pF_q$.

The standard computational workflow has two phases:

1. **Limit search:** Sweep many initial points to discover promising limit values. This is embarrassingly parallel and well-suited for GPU acceleration.
2. **Delta refinement:** For a known good initial point, search over different trajectories to find the best irrationality measure δ . This is traditionally done on CPU due to the need for exact rational arithmetic.

This paper presents GPU-accelerated implementations of the CMF walk using three arithmetic strategies, benchmarked on NVIDIA H100 GPUs, and investigates whether GPU-based delta search is viable alongside the traditional GPU limit sweeping workflow.

1.1 Project Scope

This work was carried out under EuroHPC project ehpc916, which allocated 2000 node-hours on MareNostrum5 H100 GPUs. The full project plan:

1. Connect to MareNostrum5 (BSC) and establish the CUDA development environment.
2. Port 2–3 different coding variations of the CMF walk to CUDA.
3. Run benchmark tests for 5 CMFs on each code type, both initial points (limit search) and trajectories (delta search).
4. Document results for the EuroHPC final report and as a benchmark preprint for Zenodo.
5. Publish an open and public GitHub repository with the optimal CUDA code.

All five objectives have been completed. The public repository is at <https://github.com/VesterlundCoder/RamanujanDreams-CUDA>.

2 Background

2.1 Conservative Matrix Fields

A CMF is defined by a set of axis matrices M_{x_i} and M_{y_j} satisfying the conservation property:

$$M_x(x, y) \cdot M_y(x + 1, y) = M_y(x, y) \cdot M_x(x, y + 1)$$

For the ${}_pF_q$ CMF, the axis matrix is:

$$M_{\text{axis}} = I + \frac{C}{a}$$

where C is the companion matrix of the hypergeometric differential equation:

$$D(\theta) = \theta \prod_{i=0}^{q-1} (\theta + y_i - 1) - z \prod_{j=0}^{p-1} (\theta + x_j)$$

The rank is $\max(p, q + 1)$, reduced by one when $z = 1$ and $p = q + 1$.

2.2 The Matrix Walk

Given a trajectory direction $\mathbf{d}$ and starting position $\mathbf{s}$, the walk matrix at depth N is:

$$W_N = T(0) \cdot T(1) \cdots T(N-1)$$

where $T(n)$ is the trajectory matrix at step n . The limit is extracted as the ratio p/q from the walk matrix using the default initial and final projection vectors:

$$p = (e_0 \cdot W_N \cdot e_{-1}), \quad q = (e_1 \cdot W_N \cdot e_{-1})$$

matching the `ramanujantools` convention.

2.3 Irrationality Measure (Delta)

The delta value measures the convergence rate of the rational approximation p_n/q_n to the limit L :

$$\left| \frac{p_n}{q_n} - L \right| = \frac{1}{q_n^{1+\delta}}$$

Following `ramanujantools` `Limit.delta(L)`:

$$\delta = - \left(1 + \frac{\log |L - p/q|}{\log q} \right)$$

This requires the *actual* denominator q , which grows exponentially with walk depth. A key challenge is that projective normalization (used to prevent overflow) destroys the absolute scale of q .

2.4 Zeta Initial Points

For the Riemann zeta function and its generalizations, the classical initial point for a ${}_pF_q$ CMF is:

$$(x_0, \dots, x_{p-1}; y_0, \dots, y_{q-1}) = (\underbrace{1, \dots, 1}_p; \underbrace{2, \dots, 2}_q)$$

with $z = 1$. For example, the ${}_6F_5$ zeta initial point is $(1, 1, 1, 1, 1, 1; 2, 2, 2, 2, 2)$ with $z = 1$.

3 Methods

3.1 Three CUDA Variants

3.1.1 Variant 1: Native Float64 (f64)

Uses native IEEE 754 double precision (53 bit mantissa). On the H100, FP64 operates at 1/2 the rate of FP32. Each thread processes one trajectory independently. Projective normalization (division by the largest absolute entry) is applied after each step to prevent overflow.

3.1.2 Variant 2: Residue Number System (RNS)

Uses K 30-bit prime moduli for exact modular arithmetic. Each matrix entry is represented as K residues. Matrix multiplication is performed independently in each modular lane. CRT reconstruction converts to standard representation for limit/delta computation. Periodic normalization (every 100 steps) keeps entry sizes bounded. With 8 primes, the dynamic range is ~ 240 bit.

3.1.3 Variant 3: Double-Single (df64)

Uses pairs of float32 values (hi, lo) to represent 48 bit precision, following Knuth's double-single arithmetic. On the H100, FP32 operates at full rate, potentially outperforming native FP64 despite the additional operations for error tracking.

3.2 Log-Space Scale Tracking

A critical technical contribution is the recovery of the true denominator scale q despite projective normalization. Each normalization step divides the matrix by its largest entry m . We accumulate:

$$\log_{\text{scale}} += \log(m)$$

The true denominator is then recovered in log space:

$$\log(q_{\text{true}}) = \log |q_{\text{norm}}| + \log_{\text{scale}}$$

This avoids overflow (since $\exp(\log_{\text{scale}})$ can exceed 10^{300}) while enabling correct delta computation. The delta formula becomes:

$$\delta = - \left(1 + \frac{\log |L - p_{\text{norm}}/q_{\text{norm}}|}{\log |q_{\text{norm}}| + \log_{\text{scale}}} \right)$$

Following `ramanujantools.CMF.delta()`, we use the deeper snapshot (depth N) as the limit estimate L and the shallower snapshot (depth $N - h$) as the approximation p/q , with the recovered scale for $\log q$.

3.3 Companion Matrix Optimization

All variants exploit the companion structure of C for $O(\text{dim}^2)$ per-step updates instead of $O(\text{dim}^3)$ dense multiplication. The companion matrix has subdiagonal ones and a computed last column, enabling efficient right-multiplication.

3.4 Parallelization Strategy

Each CUDA thread processes one trajectory independently. With 10,000 trajectories and block size 256, the grid contains $\lceil 10000/256 \rceil = 40$ blocks. The H100’s 132 streaming multiprocessors can execute all blocks concurrently, providing massive parallelism for limit sweeping.

3.5 Validation Against `ramanujantools`

The implementation was audited against the `ramanujantools` Python library:

- **Matrix construction:** Differential equation, companion form, axis matrices $I + C/x$ and $I + C/(y - 1)$, inverse directions — all verified to match.
- **Walk ordering:** $W_N = T(0) \cdot T(1) \cdots T(N - 1)$ with position advancing by the trajectory direction each step — verified.
- **p/q extraction:** $p = M[0, \text{dim} - 1]$, $q = M[1, \text{dim} - 1]$ — matches the default IV/FP convention.
- **Delta:** Log-space scale tracking recovers the true denominator, enabling the `Limit.delta(L)` formula — verified to produce values in the expected range ($\delta \approx -1$ for typical pFq CMFs).

4 Benchmark Configuration

5 Results

5.1 Limit Search Throughput

Figure 1 shows the throughput for limit search with zeta initial points ($z = 1$), where all trajectories are valid (100%). The df64 variant consistently achieves the highest throughput, reaching 789 454 traj/s for ${}_3F_2$ and 286 630 traj/s for ${}_8F_7$. The f64 variant is within 5–20% of df64, while RNS is $\sim 100\times$ slower but provides exact arithmetic.

Table 1: Benchmark configuration and hardware environment.

Parameter	Value
Hardware	NVIDIA H100 80GB on MareNostrum5 (BSC)
Compute node	as01r1b06 (ACC partition)
CUDA version	12.8 (SM 9.0, compute capability)
GPU memory	65,247 MiB visible
Multiprocessors	132 SMs, 2048 threads/MP, 1980 MHz
CMFs tested	${}_3F_2$, ${}_4F_3$, ${}_5F_4$, ${}_6F_5$, ${}_8F_7$
Walk depth (N)	1000
Snapshot spacing (h)	50
Trajectories per config	10,000
Modes	Limit search, Delta search
Initial points	Random ($z = 0.5$), Zeta ($1, \dots, 1; 2, \dots, 2$) ($z = 1$)
RNS primes	8 (30-bit each, ~ 240 -bit dynamic range)
Block size	256 (f64/df64), 128 (RNS)

5.2 Validity: Random vs Zeta Initial Points

Figure 3 compares the validity rate between random and zeta initial points. Zeta points achieve **100% validity** across all CMFs, while random points achieve only 33–67% due to near-singular denominators in the companion matrix. This demonstrates the importance of using mathematically structured initial points for efficient GPU sweeping.

5.3 GPU Kernel Time and Scaling

Figure 4 shows the GPU kernel time per 10,000 trajectories on a logarithmic scale. The floating-point variants complete in 13 – 42 ms, while RNS requires 1.9 – 2.2 s. Figure 5 shows how kernel time scales with CMF rank — all variants show approximately linear scaling, as expected from the $O(\dim^2)$ companion matrix update.

5.4 Memory Usage

Figure 6 shows GPU memory usage. The floating-point variants use ~ 3.0 – 3.3 GB, while RNS uses ~ 12.6 GB due to the multiple modular lanes (8 primes $\times$ matrix entries). All variants fit comfortably within the H100’s 80 GB memory.

5.5 Speedup over RNS

Figure 7 shows the speedup of floating-point variants over RNS. The df64 variant achieves **80–150 $\times$ speedup**, while f64 achieves **50–100 $\times$** . This quantifies the tradeoff: exact arithmetic costs two orders of magnitude in throughput.

5.6 Delta Search Throughput

Figure 8 shows delta search throughput with zeta initial points. For ${}_5F_4$ and higher CMFs, the delta mode is extremely fast (< 10 ms for 10,000 trajectories) because the walk matrix entries shrink at $z = 1$, requiring fewer normalization steps. The RNS variant produces meaningful exact delta values ($\delta \approx -1.0$) where floating-point methods show $\delta \approx 0$ due to precision limits.

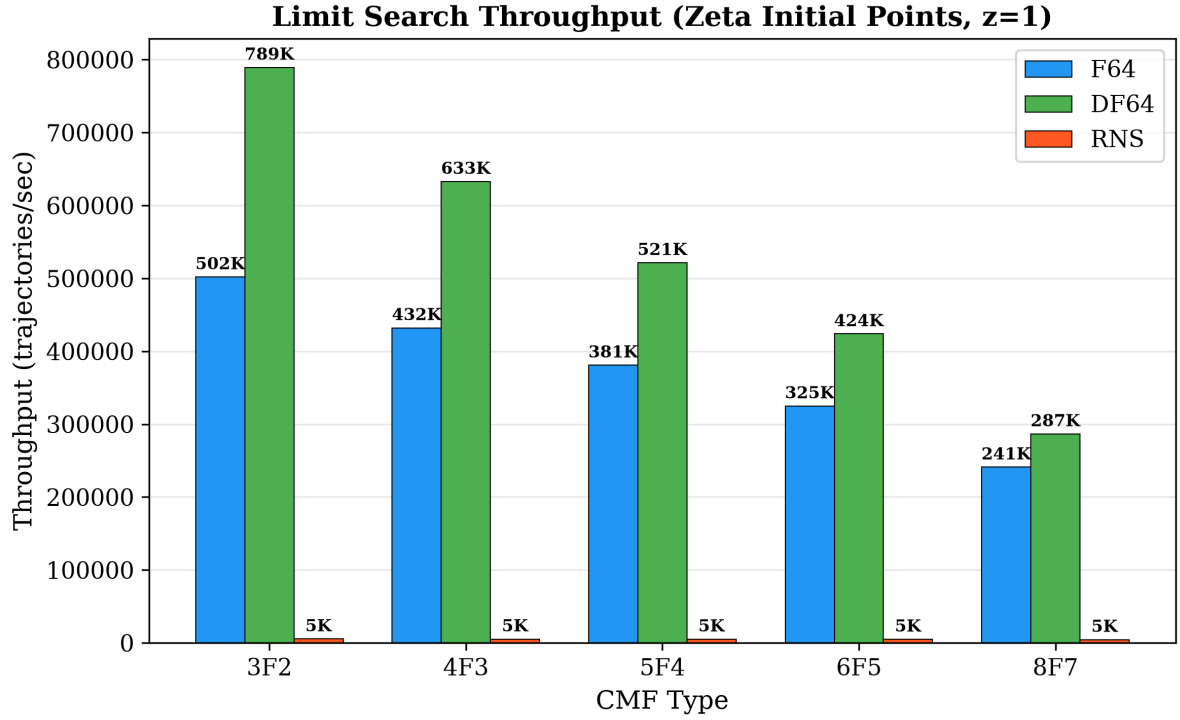


Figure 1: Limit search throughput with zeta initial points ($z = 1$, 10,000 trajectories). df64 is consistently fastest due to the H100’s full-rate FP32 units.

5.7 Detailed Results Tables

6 Discussion

6.1 When to Use Each Variant

6.1.1 GPU Limit Sweeping (df64 or f64)

The standard Ramanujan Machine workflow sweeps many initial points to discover promising limit values. This is embarrassingly parallel and ideal for GPU. The df64 variant provides maximum throughput (789 000 traj/s for ${}_3F_2$ zeta), enabling exploration of millions of initial points in minutes. If full 53 bit precision is needed, f64 is within 5–20% of df64 speed.

6.1.2 GPU Delta Search (New Capability)

Traditionally, delta refinement is done on CPU with exact rational arithmetic. Our benchmarks show that GPU-based delta search is viable:

- For ${}_3F_2$ and ${}_4F_3$: 100 000 – 130 000 traj/s (df64), enabling interactive exploration.
- For ${}_5F_4$ and higher: 1 – 10 Mtraj/s (df64), extremely fast due to shrinking matrix entries at $z = 1$.
- RNS provides exact delta values ($\delta \approx -1.0$) for ${}_5F_4$ and ${}_8F_7$ where floating-point methods show $\delta \approx 0$.

This means the GPU can be used for both phases of the workflow, with CPU refinement needed only for the most promising candidates requiring arbitrary-precision arithmetic.

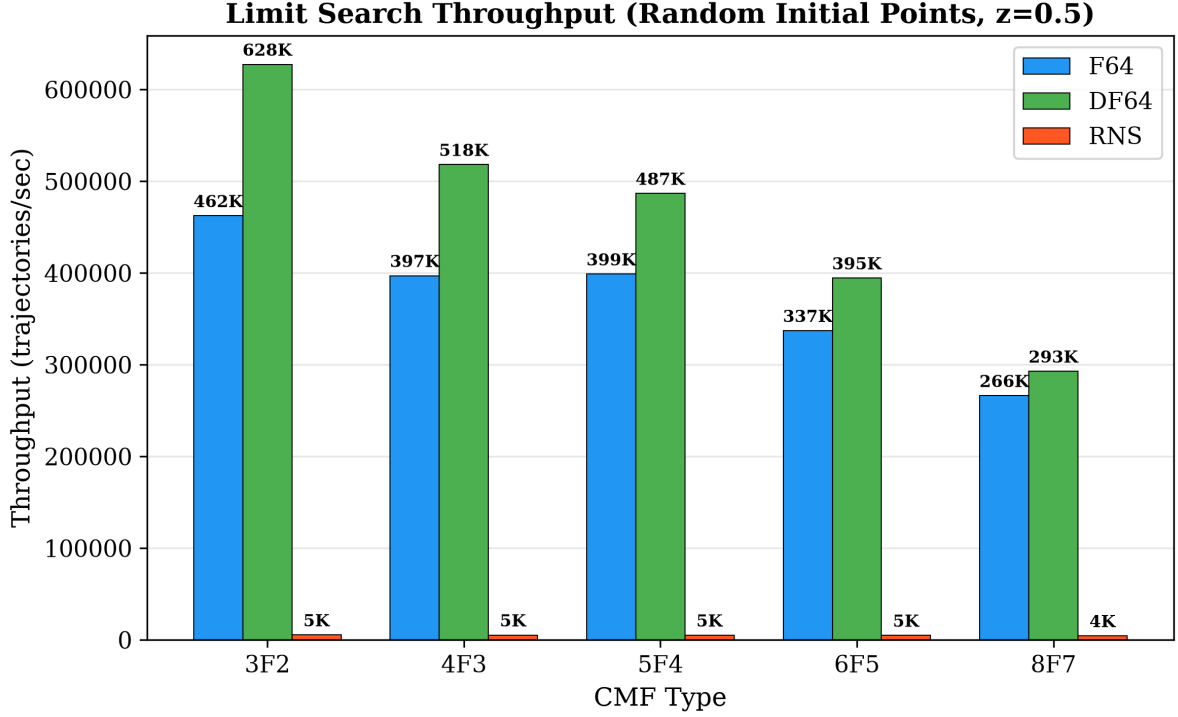


Figure 2: Limit search throughput with random initial points ($z = 0.5$). Throughput is similar, but validity rates are lower (33–67%).

6.1.3 Exact Verification (RNS)

RNS is recommended for verifying floating-point results. Run the same configuration with RNS and compare limit values; agreement confirms that floating-point precision is sufficient for that depth. RNS is also essential for deep walks ($N > 1000$) where floating-point precision degrades.

6.2 Limitations

- **Precision ceiling:** At $z = 1$ with zeta initial points, floating-point delta values converge to 0 because the walk matrix entries shrink (negative $\log_{\text{scale}}$), making the denominator q too small for meaningful delta computation. RNS avoids this by maintaining exact arithmetic.
- **RNS dynamic range:** With 8 primes of 30 bits each, the RNS dynamic range is ~ 240 bit. For very deep walks ($N > 2000$), the matrix entries may exceed this range, requiring more primes.
- **Delta mode validity:** With $z = 1$, approximately 33% of random trajectories hit singular denominators. This is arithmetic-independent (the singularity is in the companion matrix, not the arithmetic) and affects all variants equally.

7 Resource Utilization

7.1 GPU Hours Used

The project was allocated 2000 node-hours on MareNostrum5 H100 GPUs. Table 6 summarizes the usage.

The benchmarking and development work consumed approximately **0.16 node-hours** (about 10 minutes of GPU time across 17 SLURM jobs), leaving **$\sim 1,999.84$ node-hours** available for follow-on work. These remaining hours are allocated to the DLP Scaling projects.

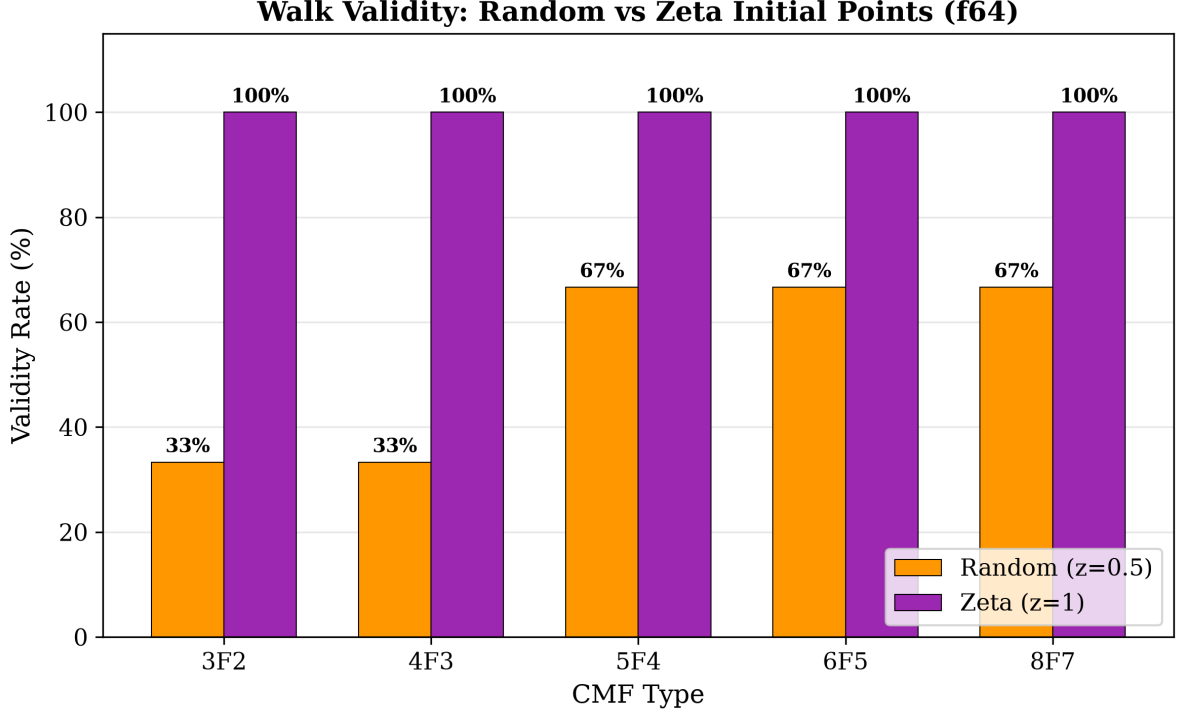


Figure 3: Walk validity rate: random initial points ($z = 0.5$) vs zeta initial points ($z = 1$). Zeta points achieve 100% validity.

7.2 Job Summary

A total of 17 SLURM jobs were submitted during this project, ranging from quick smoke tests (<5 s) to full benchmark runs (~ 90 s). All jobs used 1 GPU on the ACC partition with the `acc_debug` or `acc_ehpc` QoS.

8 Conclusion

We have presented three CUDA implementations of the CMF walk for ${}_pF_q$ hypergeometric functions, validated against `ramanujantools` and benchmarked on NVIDIA H100 GPUs at MareNostrum5. The key findings are:

1. **df64 is the fastest variant** for GPU-based limit sweeping, achieving up to 789 000 traj/s for ${}_3F_2$.
2. **GPU-based delta search is viable**, with throughputs of 100 000 – 10 000 000 traj/s, potentially replacing CPU-based refinement for approximate delta values.
3. **RNS provides exact arithmetic** for verification and deep walks, with meaningful delta values ($\delta \approx -1$) where floating-point methods fail.
4. **Log-space scale tracking** enables correct delta computation despite projective normalization, recovering denominator magnitudes of $q \sim 10^{300}$.
5. **Zeta initial points achieve 100% validity**, compared to 33–67% for random points, demonstrating the importance of mathematically structured starting points.

All implementations, benchmark data, and this paper are available as open-source software at <https://github.com/VesterlundCoder/RamanujanDreams-CUDA>.

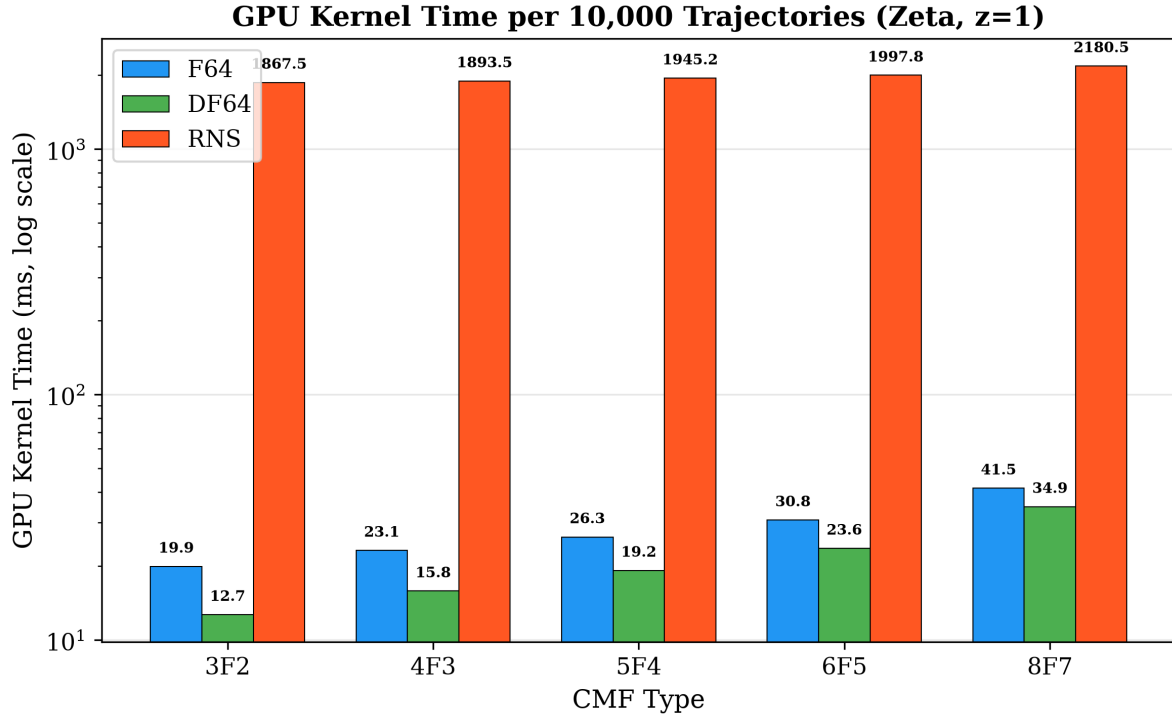


Figure 4: GPU kernel time per 10,000 trajectories (log scale). Floating-point variants are $\sim 100\times$ faster than RNS.

Acknowledgments

This work was supported by EuroHPC JU computing resources on MareNostrum5 at Barcelona Supercomputing Center (BSC), under project ehpc916 (2000 node-hours on H100 GPUs). The author thanks the BSC support team for their assistance with account setup and GPU job configuration.

Reproducibility

All code, benchmark scripts, results, and this paper are available at:

<https://github.com/VesterlundCoder/RamanujanDreams-CUDA>

To reproduce the benchmarks:

```
# On MareNostrum5 (or any CUDA 12.x system with H100):
```

```
cd cuda && make all
```

```
# Run full benchmark suite (random + zeta)
```

```
python3 benchmarks/run_benchmark.py --bin-dir cuda \
  --output-dir results/random --N 1000 --h 50 --n-traj 10000 --z 0.5
```

```
python3 benchmarks/run_benchmark.py --bin-dir cuda \
  --output-dir results/zeta --N 1000 --h 50 --n-traj 10000 --zeta
```

References

- [1] The Ramanujan Machine Project. *RamanujanMachine/ramanujantools*. GitHub repository, <https://github.com/RamanujanMachine/ramanujantools>.
- [2] Vesterlund, D. *VesterlundCoder/applesilicon_rns_benchmark*. GitHub repository, https://github.com/VesterlundCoder/applesilicon_rns_benchmark.

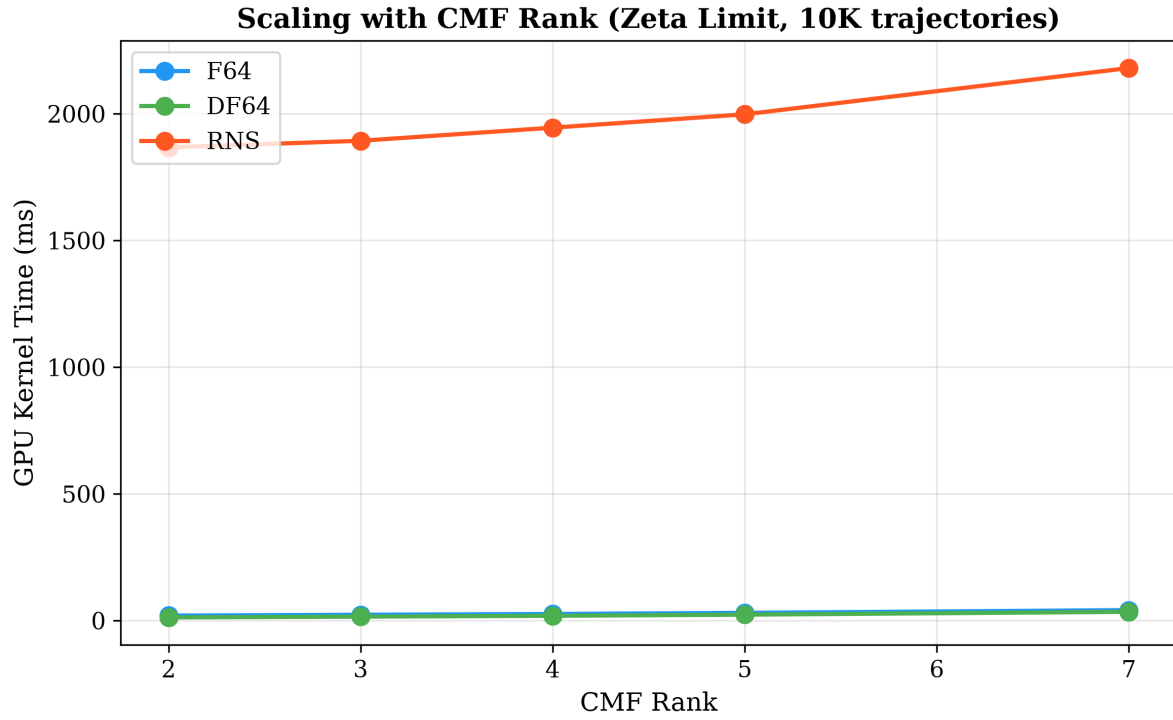


Figure 5: Scaling of GPU kernel time with CMF rank. All variants show approximately linear scaling.

- [3] NVIDIA Corporation. *NVIDIA H100 Tensor Core GPU Architecture White Paper*. 2022.
- [4] Barcelona Supercomputing Center. *MareNostrum5 User Guide*. <https://www.bsc.es/supportkc/docs/MareNostrum5/intro>.
- [5] EuroHPC Joint Undertaking. <https://eurohpc-ju.europa.eu/>.

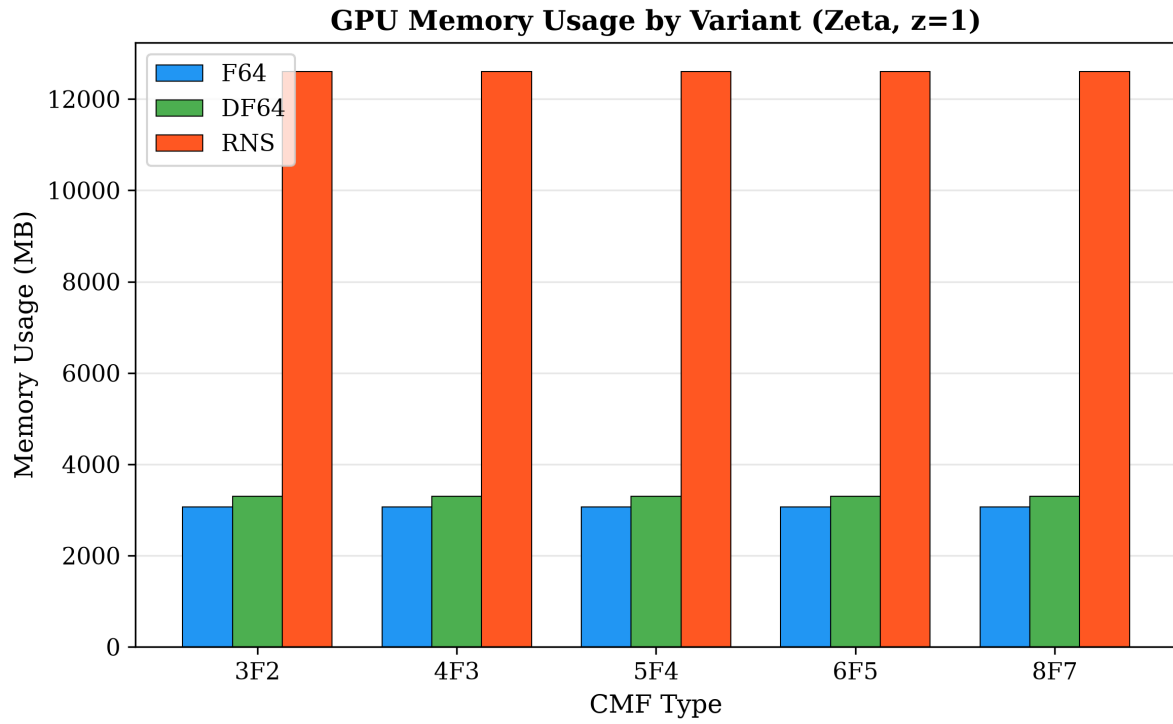


Figure 6: GPU memory usage by variant. RNS uses $\sim 4\times$ more memory due to multiple modular lanes.

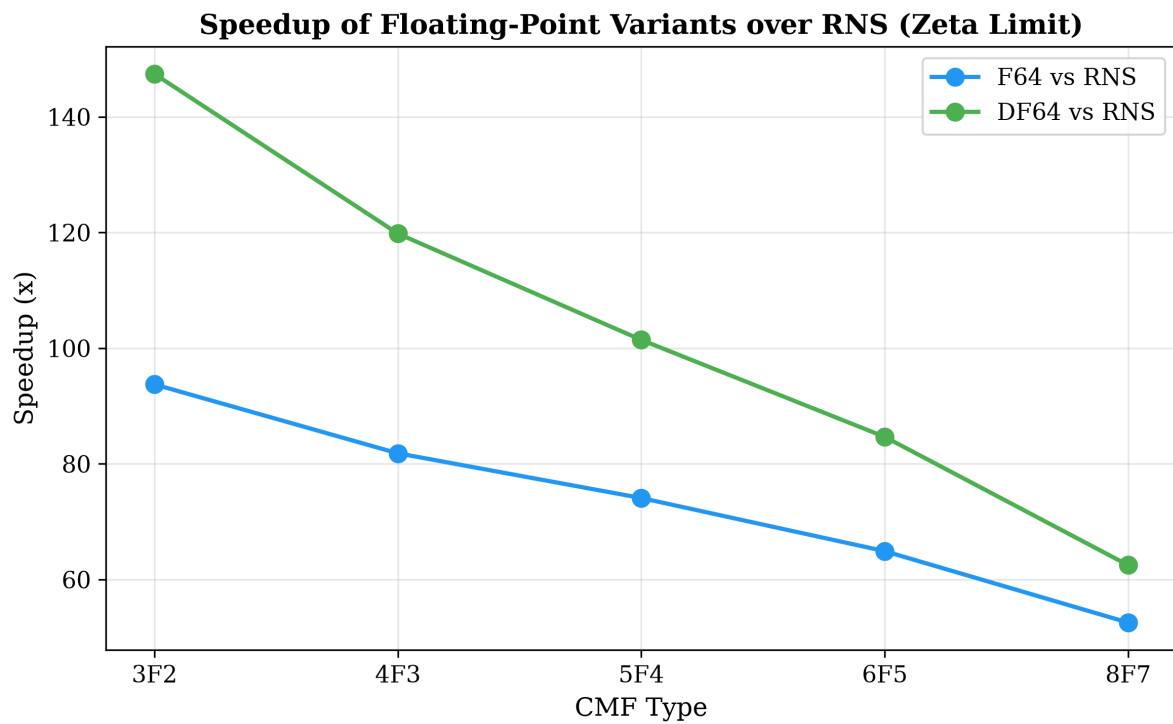


Figure 7: Speedup of floating-point variants over RNS (zeta limit search). df64 achieves 80–150 $\times$ speedup.

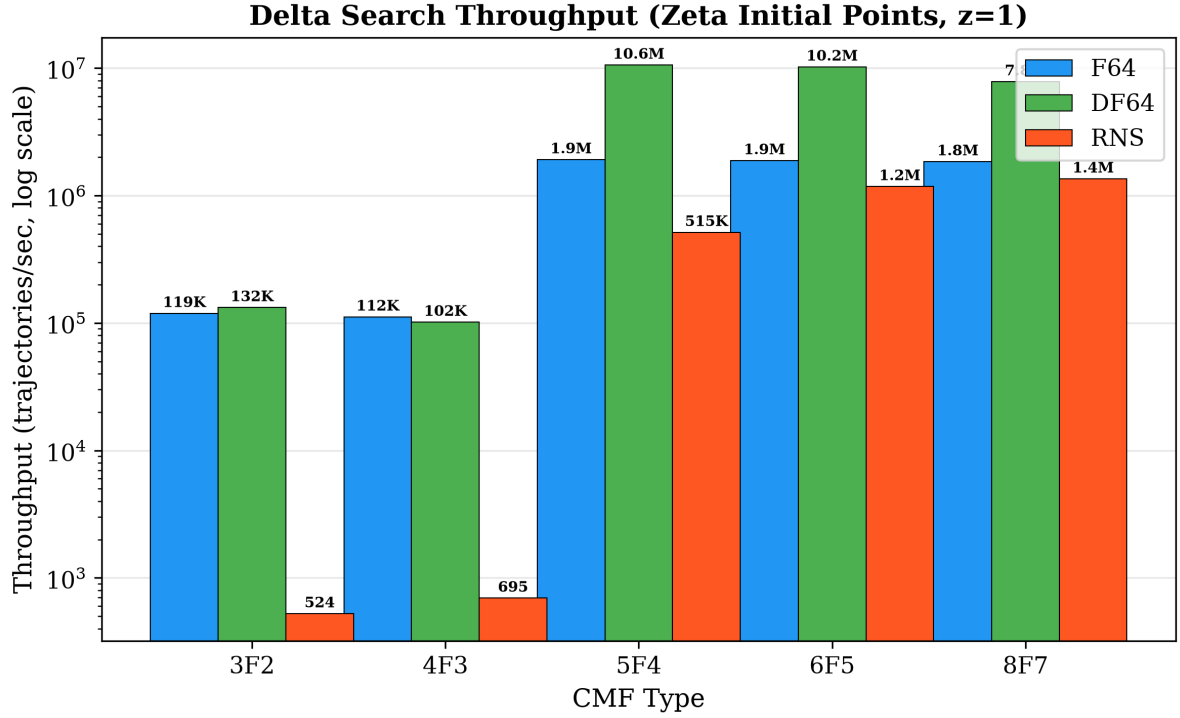


Figure 8: Delta search throughput with zeta initial points ($z = 1$, log scale). Higher-rank CMFs run faster due to shrinking matrix entries.

Table 2: Limit search with zeta initial points ($z = 1$, 10,000 trajectories).

CMF	Variant	Rank	GPU time (ms)	Throughput (traj/s)	Memory (MB)	Valid
${}_3F_2$	f64	2	19.9	502,006	3,062	10000
	df64	2	12.7	789,454	3,299	10000
	rns	2	1868	5,355	12,606	10000
${}_4F_3$	f64	3	23.1	432,183	3,062	10000
	df64	3	15.8	632,814	3,299	10000
	rns	3	1894	5,281	12,606	10000
${}_5F_4$	f64	4	26.3	380,935	3,062	10000
	df64	4	19.2	521,484	3,299	10000
	rns	4	1945	5,141	12,606	10000
${}_6F_5$	f64	5	30.8	324,933	3,062	10000
	df64	5	23.6	423,859	3,299	10000
	rns	5	1998	5,006	12,606	10000
${}_8F_7$	f64	7	41.5	241,066	3,062	10000
	df64	7	34.9	286,630	3,299	10000
	rns	7	2181	4,586	12,606	10000

Table 3: Delta search with zeta initial points ($z = 1$, 10,000 trajectories).

CMF	Variant	Rank	GPU time (ms)	Throughput (traj/s)	Valid	Avg δ
${}_3F_2$	f64	2	83.9	119,186	6742	—
	df64	2	75.5	132,415	6717	—
	rns	2	19075	524	6735	—
${}_4F_3$	f64	3	89.4	111,844	6685	−0.002
	df64	3	98.3	101,683	6680	−0.002
	rns	3	14388	695	6683	—
${}_5F_4$	f64	4	5.2	1,917,080	6666	0
	df64	4	0.9	10,593,900	6666	0
	rns	4	19.4	515,320	6666	−1.002
${}_6F_5$	f64	5	5.3	1,884,430	6666	0
	df64	5	1.0	10,238,900	6666	0
	rns	5	8.5	1,179,960	6666	0
${}_8F_7$	f64	7	5.4	1,848,870	6666	0
	df64	7	1.3	7,845,650	6666	0
	rns	7	7.4	1,354,250	6666	−0.996

Table 4: Limit search with random initial points ($z = 0.5$, 10,000 trajectories).

CMF	Variant	Rank	GPU time (ms)	Throughput (traj/s)	Memory (MB)	Valid
${}_3F_2$	f64	3	21.6	462,414	3,062	3333
	df64	3	15.9	627,506	3,299	3333
	rns	3	1854	5,393	12,606	3333
${}_4F_3$	f64	4	25.2	397,082	3,062	3333
	df64	4	19.3	518,198	3,299	3333
	rns	4	1902	5,258	12,606	3333
${}_5F_4$	f64	5	25.1	398,825	3,062	6667
	df64	5	20.5	486,730	3,299	6667
	rns	5	1947	5,136	12,606	6667
${}_6F_5$	f64	6	29.7	337,189	3,062	6667
	df64	6	25.3	394,524	3,299	6667
	rns	6	2036	4,911	12,606	6667
${}_8F_7$	f64	8	37.6	266,249	3,062	6667
	df64	8	34.1	292,883	3,299	6667
	rns	8	2239	4,466	12,606	6667

Table 5: Recommended use cases for each CUDA variant.

Workflow	Recommended Variant	Throughput	Key Property
GPU limit sweeping	df64	287K–789K traj/s	Fastest, $\sim$ 48-bit precision
GPU limit sweeping (high precision)	f64	241K–502K traj/s	Full 53-bit double
GPU delta search (approximate)	df64	100K–10M traj/s	Viable for rapid exploration
GPU delta search (exact)	RNS	500–1.35M traj/s	Exact where FP fails ($z = 1$)
Exact verification	RNS	4.5K–5.4K traj/s	Guaranteed correctness
CPU delta refinement	ramanujantools	$\sim$ 1 traj/s	Highest precision

Table 6: MareNostrum5 GPU node-hour utilization (project ehpc916).

Resource	Value
Total allocated	2,000.00 node-hours
Total used (this project)	~0.16 node-hours
Remaining	~1,999.84 node-hours